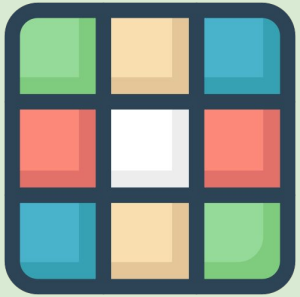
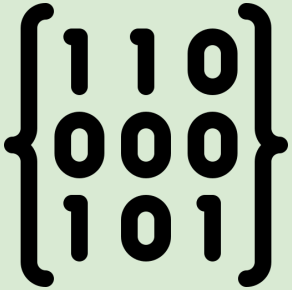
Lesson 5
Arrays, String





agilaghamirzayev

Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets [ ]** :

Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets** **[]** :

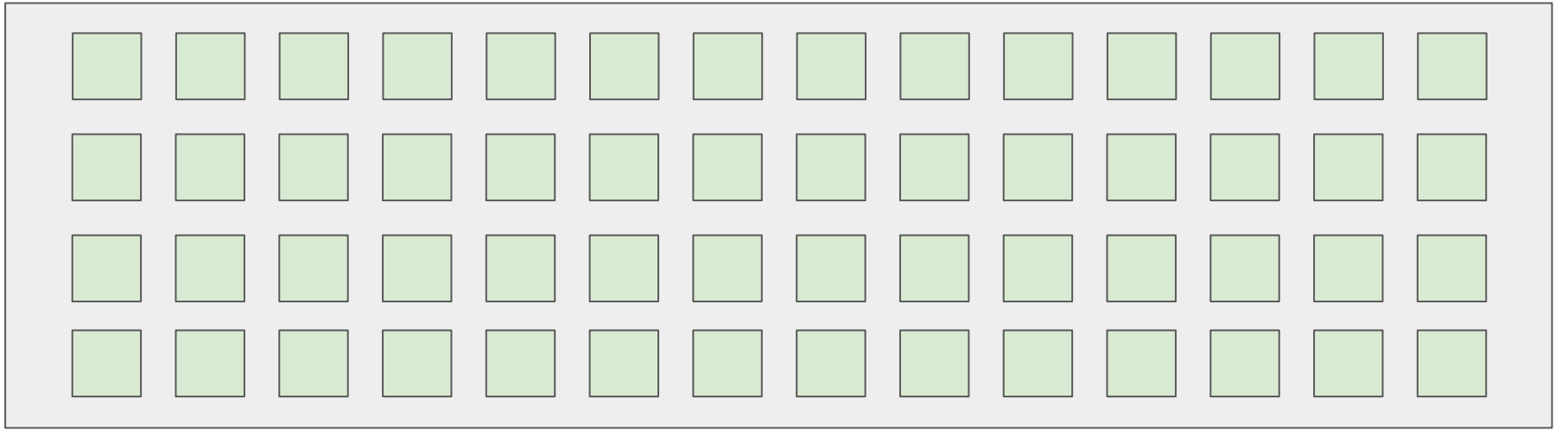
```
int[] nums = { 1, 2, 3, 4, 5}; // correct
int nums[] = { 1, 2, 3, 4, 5}; // correct
int[] nums = new int[5];       //correct
```

Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets** **[]** :

```
int num = 5;
int[] nums = new int[5];
```

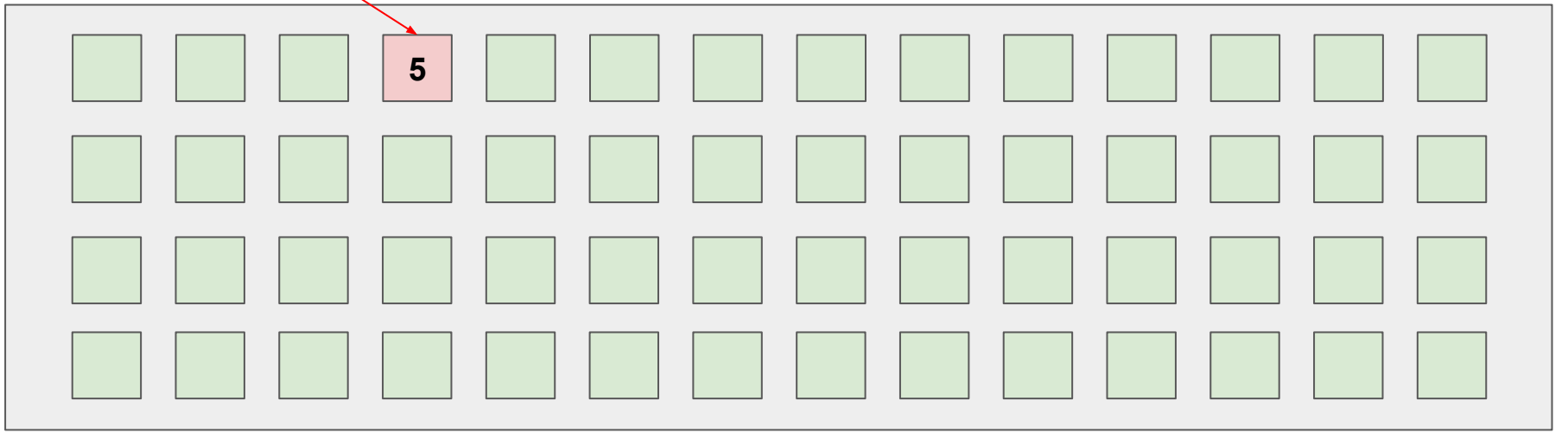


Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets** **[ ]** :

```
int num = 5;
int[] nums = new int[5];
```

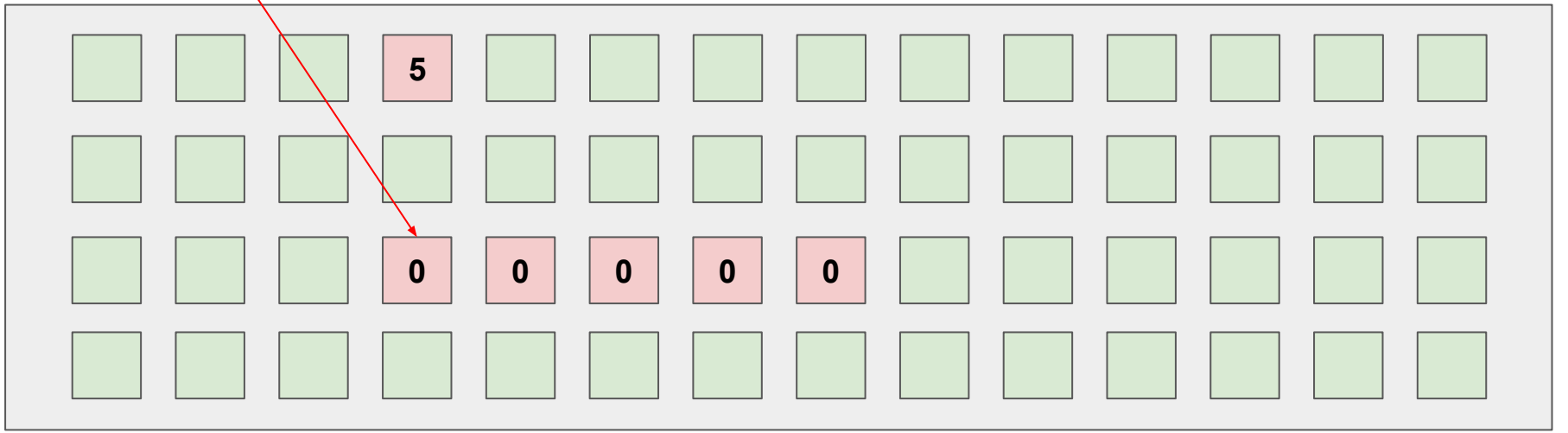


Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets** **[ ]** :

```
int num = 5;
int[] nums = new int[5];
```

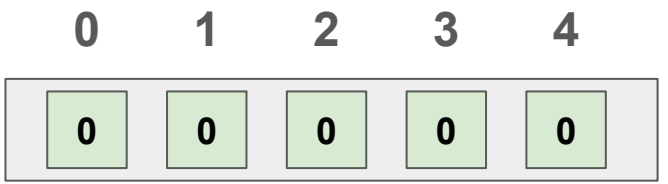


Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets** **[ ]** :

```
int[] nums = new int[5];        //correct
```



Java Arrays

Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets [ ]** :

```java
int[] nums = new int[5]; //correct
```

```java
int[] nums = { 1, 2, 3, 4, 5}; // correct
```

```java
nums[0] = 1;
nums[1] = 2;
nums[2] = 3;
nums[3] = 4;
nums[4] = 5;
```

Java Arrays

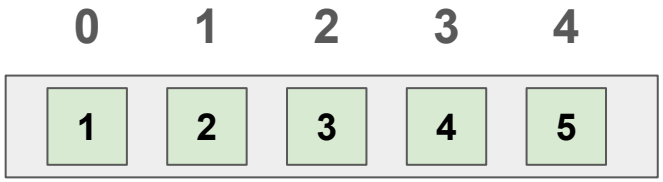
Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets [ ]** :

```
int[] nums = new int[5]; //correct
```

```
int[] nums = { 1, 2, 3, 4, 5}; // correct
```

```
nums[0] = 1;
nums[1] = 2;
nums[2] = 3;
nums[3] = 4;
nums[4] = 5;
```



Java Arrays

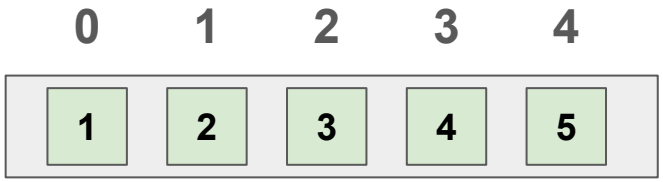
Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets [ ]** :

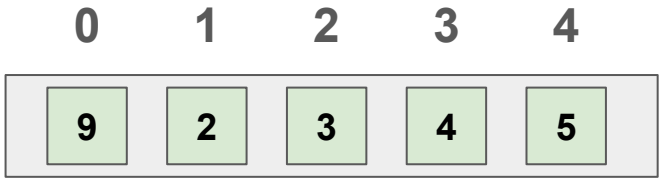
```
int[] nums = new int[5]; //correct
```

```
int[] nums = { 1, 2, 3, 4, 5}; // correct
```

```
nums[0] = 1;
nums[1] = 2;
nums[2] = 3;
nums[3] = 4;
nums[4] = 5;
```




```
nums[0] = 9;
```



agilaghamirzayev

Java Arrays

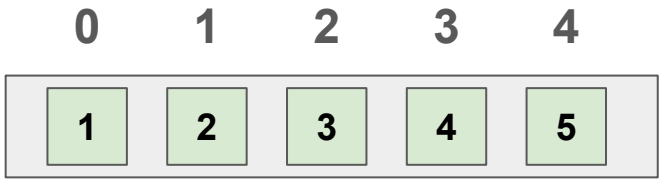
Arrays are used to store **multiple values in a single variable**, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets [ ]** :

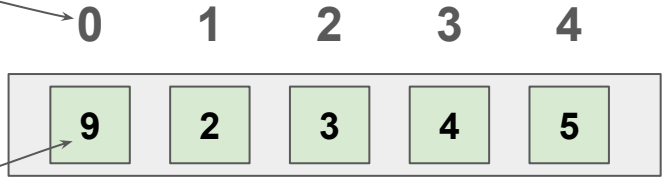
```
int[] nums = new int[5]; //correct
```

```
int[] nums = { 1, 2, 3, 4, 5}; // correct
```

```
nums[0] = 1;
nums[1] = 2;
nums[2] = 3;
nums[3] = 4;
nums[4] = 5;
```




```
nums[0] = 9;
```



agilaghamirzayev

DEMO

Java Arrays Loops

```java
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (int i = 0; i < cars.length; i++) {
  System.out.println(cars[i]);
}
```

```java
int[] numbers = {10, 20, 30, 40};

for (int i = 0; i < numbers.length; i++) {
  System.out.println(numbers[i]);
}
```

Java Arrays Loops

```java
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (int i = 0; i < cars.length; i++) {
  System.out.println(cars[i]);
}
```

```java
int[] numbers = {10, 20, 30, 40};

for (int i = 0; i < numbers.length; i++) {
  System.out.println(numbers[i]);
}
```

1. An array storing different ages
2. Calculate the average and sum of all ages
3. Find the lowest age
4. Sort the ages from lowest to highest

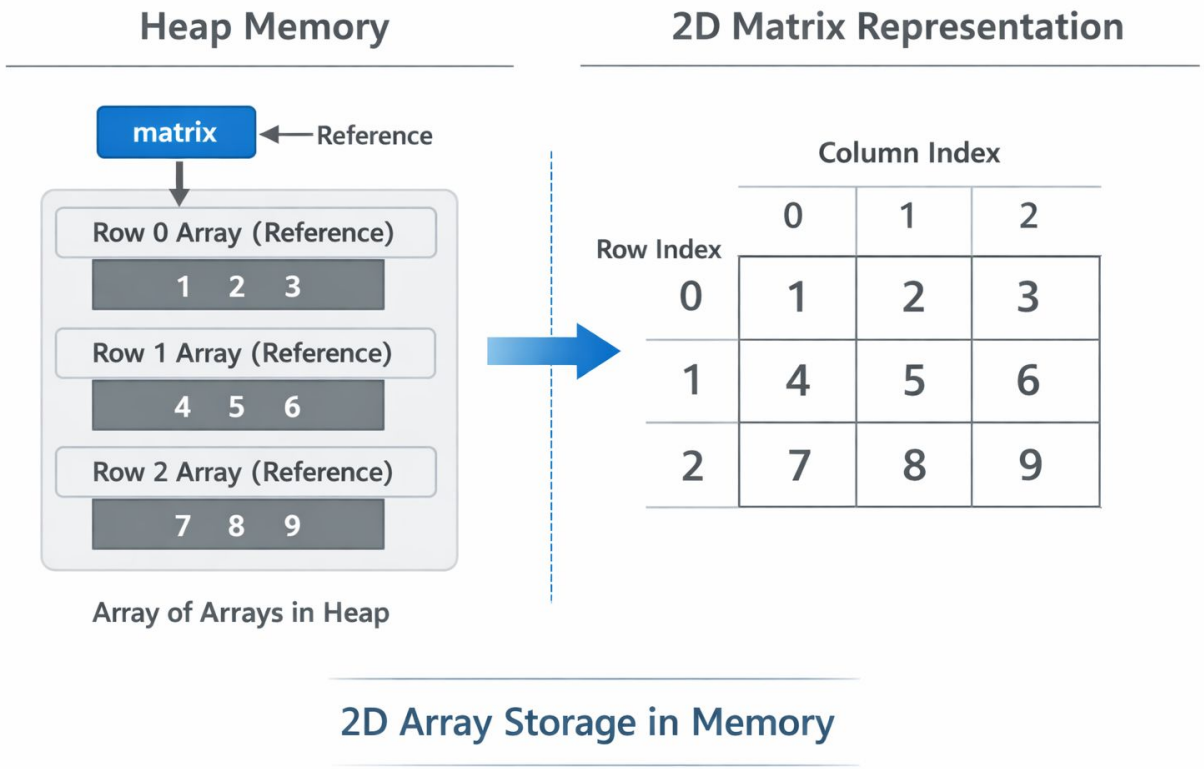
https://visualgo.net/en/sorting

```java
public class AgesAnalysis {

    public static void main(String[] args) {

        int[] ages = {18, 25, 30, 16, 22, 40, 19};

        int sum = calculateSum(ages);
        double average = calculateAverage(ages);
        int min = findMinimum(ages);
        int[] sortedAges = sortAges(ages.clone()); // clone to keep original
safe
        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
        System.out.println("Minimum age: " + min);

        System.out.print("Sorted ages: ");
        for (int age : sortedAges) {
            System.out.print(age + " ");
        }
    }
}
```

DEMO

Java 2D Arrays

```java
version 1:
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

version 2:
int[][] matrix = new int[3][3];
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[0][2] = 3;

matrix[1][0] = 4;
matrix[1][1] = 5;
matrix[1][2] = 6;

matrix[2][0] = 7;
matrix[2][1] = 8;
matrix[2][2] = 9;
```



agilaghamirzayev

Java 2D Arrays Loops

```java
int[][] myNumbers = { {1, 4, 2}, {3, 6, 8, 5, 2} };

System.out.println("Rows: " + myNumbers.length);             // 2
System.out.println("Cols in row 0: " + myNumbers[0].length); // 3
System.out.println("Cols in row 1: " + myNumbers[1].length); // 5
```

```java
int[][] myNumbers = { {1, 4, 2}, {3, 6, 8, 5, 2} };

for (int row = 0; row < myNumbers.length; row++) {
  for (int col = 0; col < myNumbers[row].length; col++) {
    System.out.println("myNumbers[" + row + "][" + col + "] = " + myNumbers[row][col]);
  }
}
```

DEMO

Java String

A `String` variable contains a collection of characters surrounded by double quotes (`""`):

```
String name = "Fluffy";
```

Java String

A `String` variable contains a collection of characters surrounded by double quotes (`""`):

```
String name = "Fluffy";
String name = new String("Fluffy");
```

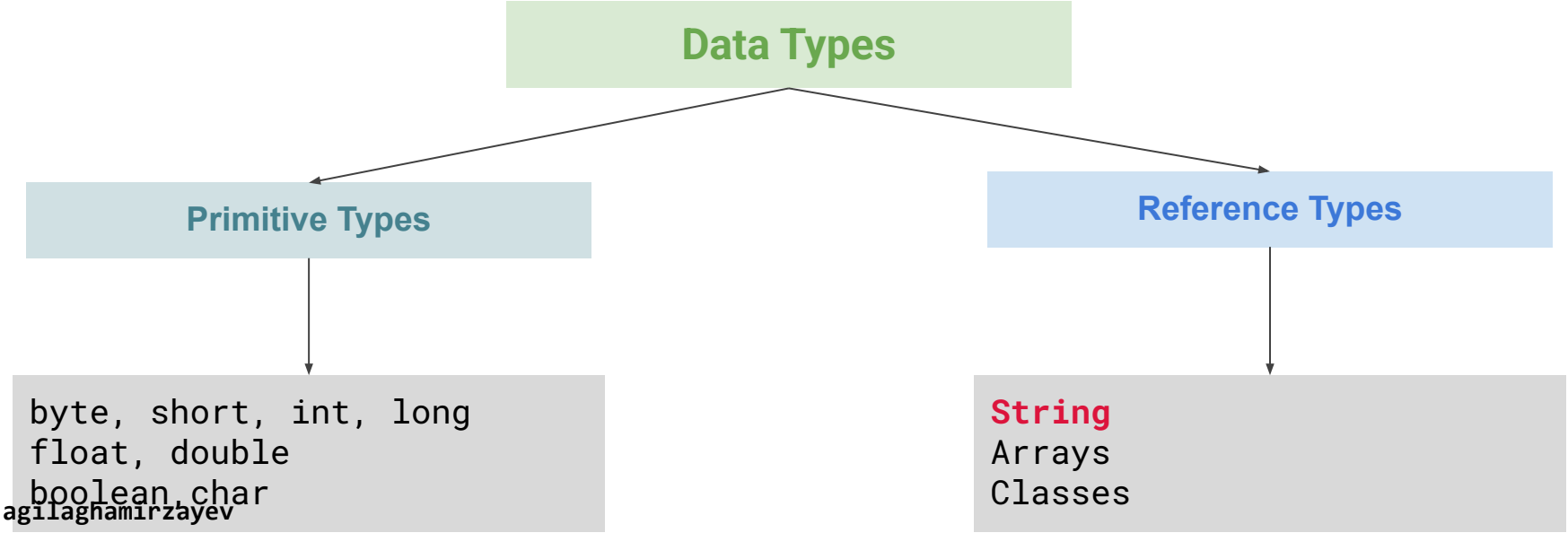
Reference types are created using the **new** keyword

Java String

A `String` variable contains a collection of characters surrounded by double quotes (`""`):

```
String name = "Fluffy";
String name = new String("Fluffy");
```

Reference types are created using the **new** keyword



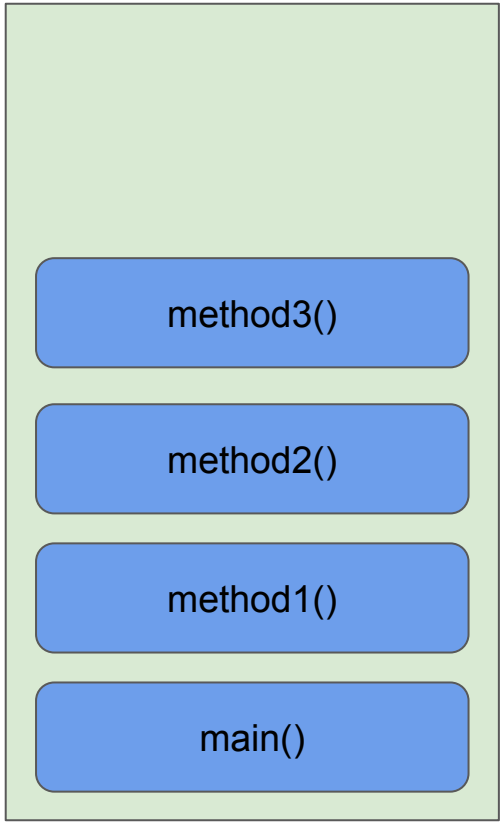
agilaghamirzayev

Java String

A `String` variable contains a collection of characters surrounded by double quotes (`""`):

```
String name = "Fluffy";
String name = new String("Fluffy");
```

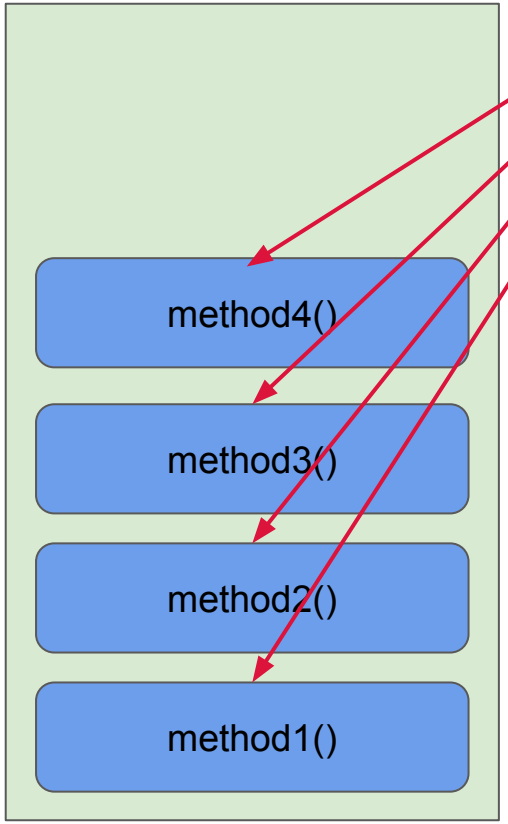
STACK



LIFO - last in first out

in agilaghamirzayev

STACK



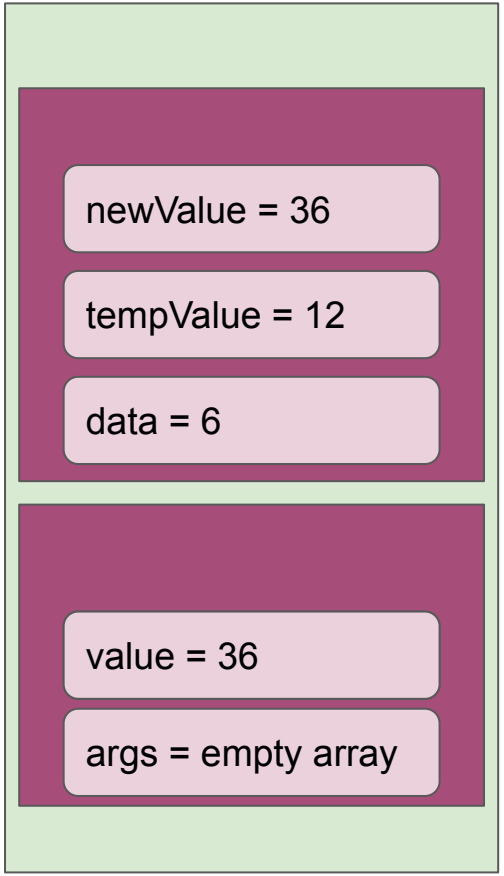
LIFO - last in first out

in agilaghamirzayev

agilaghamirzayev

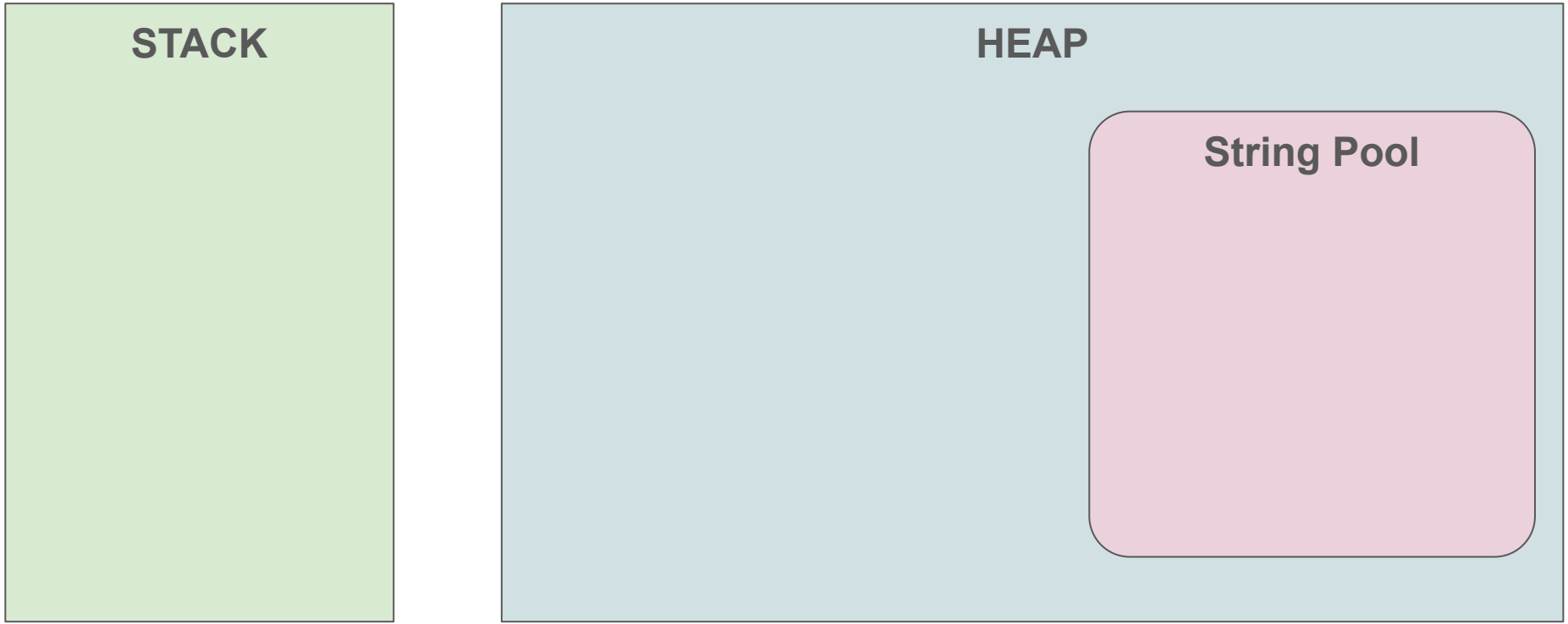
```java
public class Main {

  public static void main(String[] args) {
    int value = 6;

    value = operate(value);
  }

  private static int operate(int data) {
    int tempValue = data + 6;
    int newValue = tempValue * 3;
    return newValue;
  }
}
```

STACK



Java String

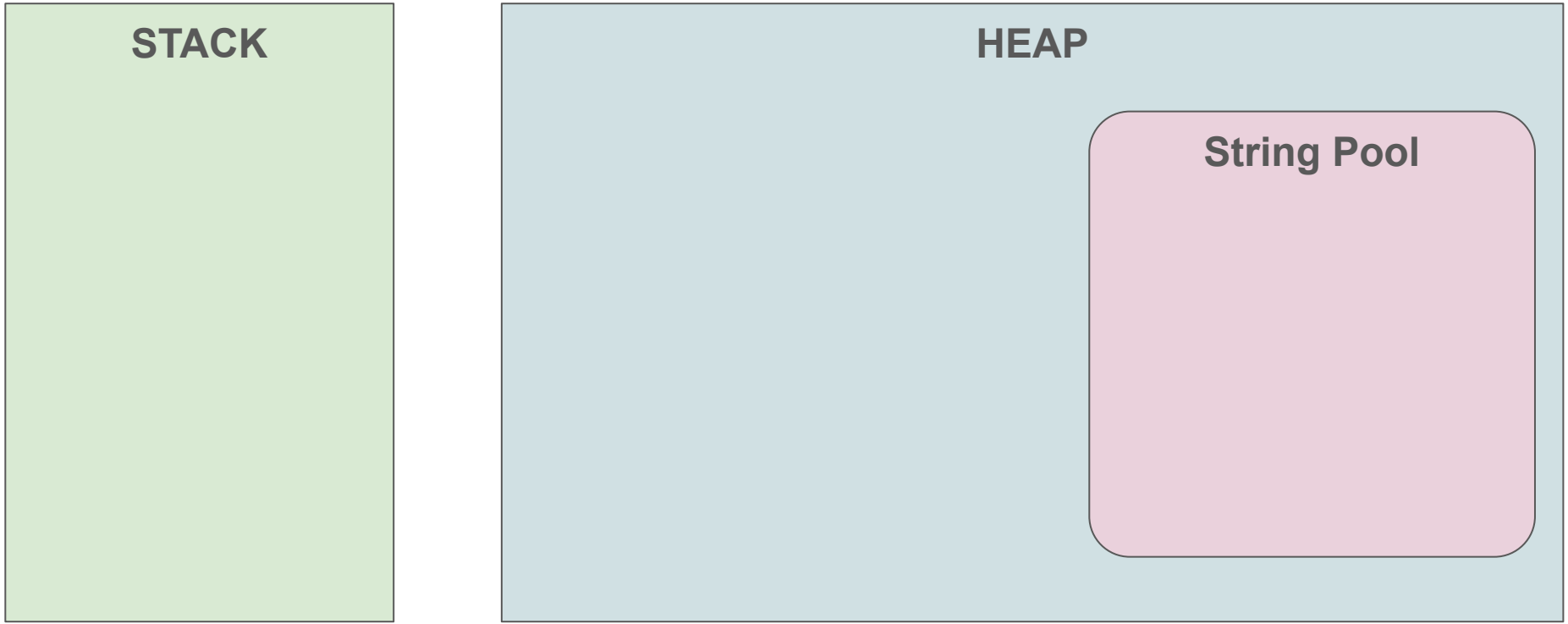
in agilaghamirzayev



Java String

agilaghamirzayev

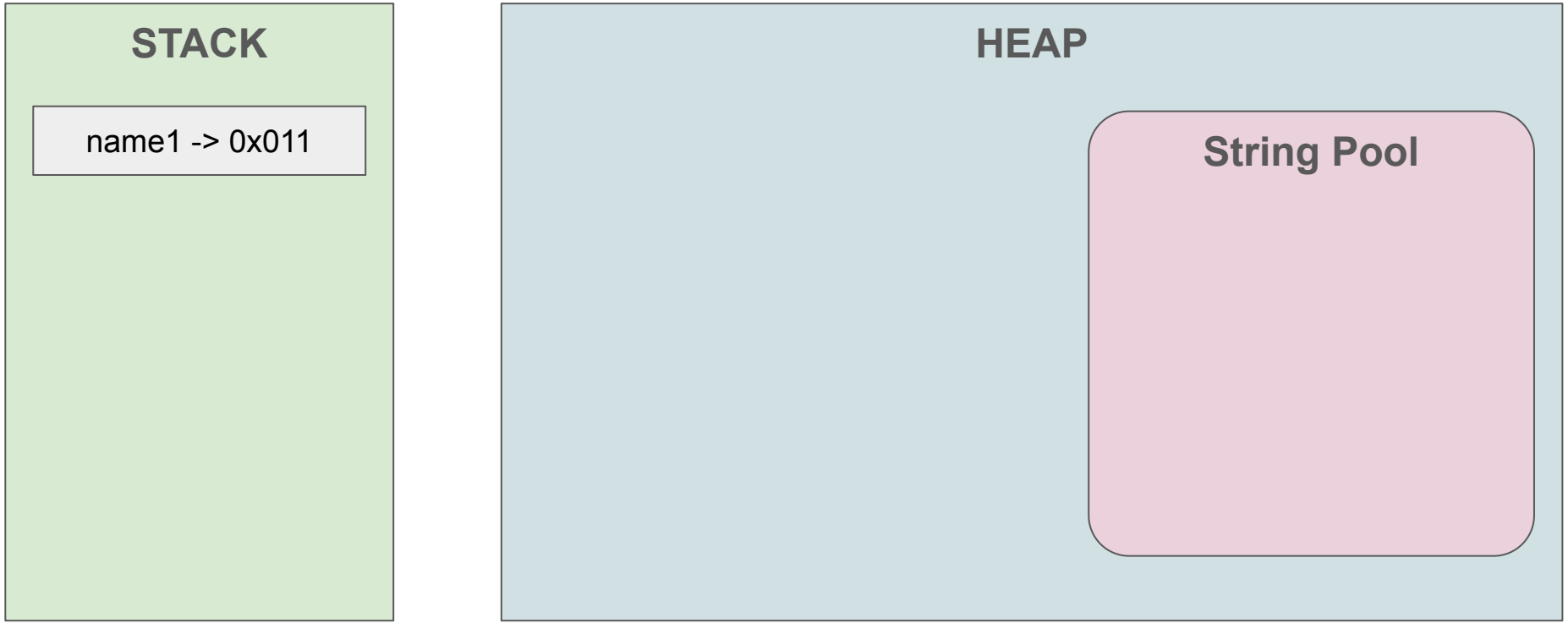
```
String name1 = new String("Test");
```



Java String

agilaghamirzayev

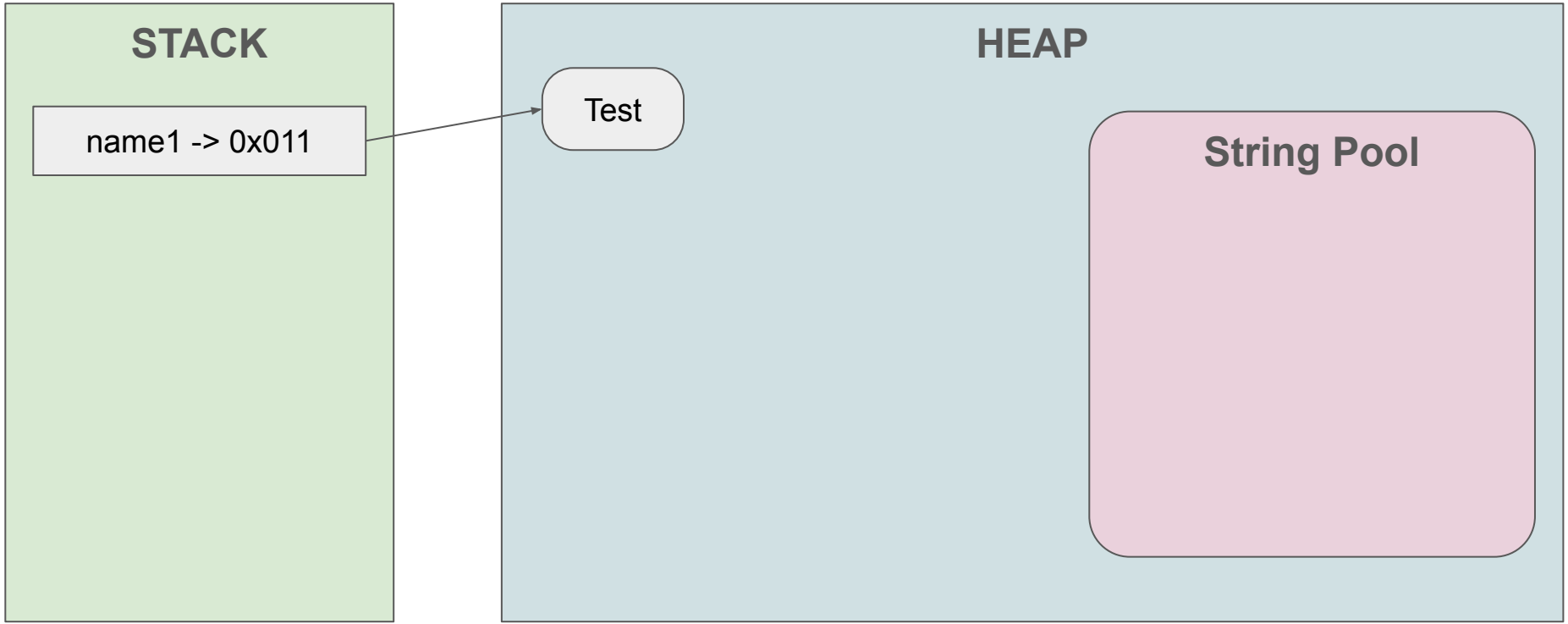
```
String name1 = new String("Test");
```



Java String

agilaghamirzayev

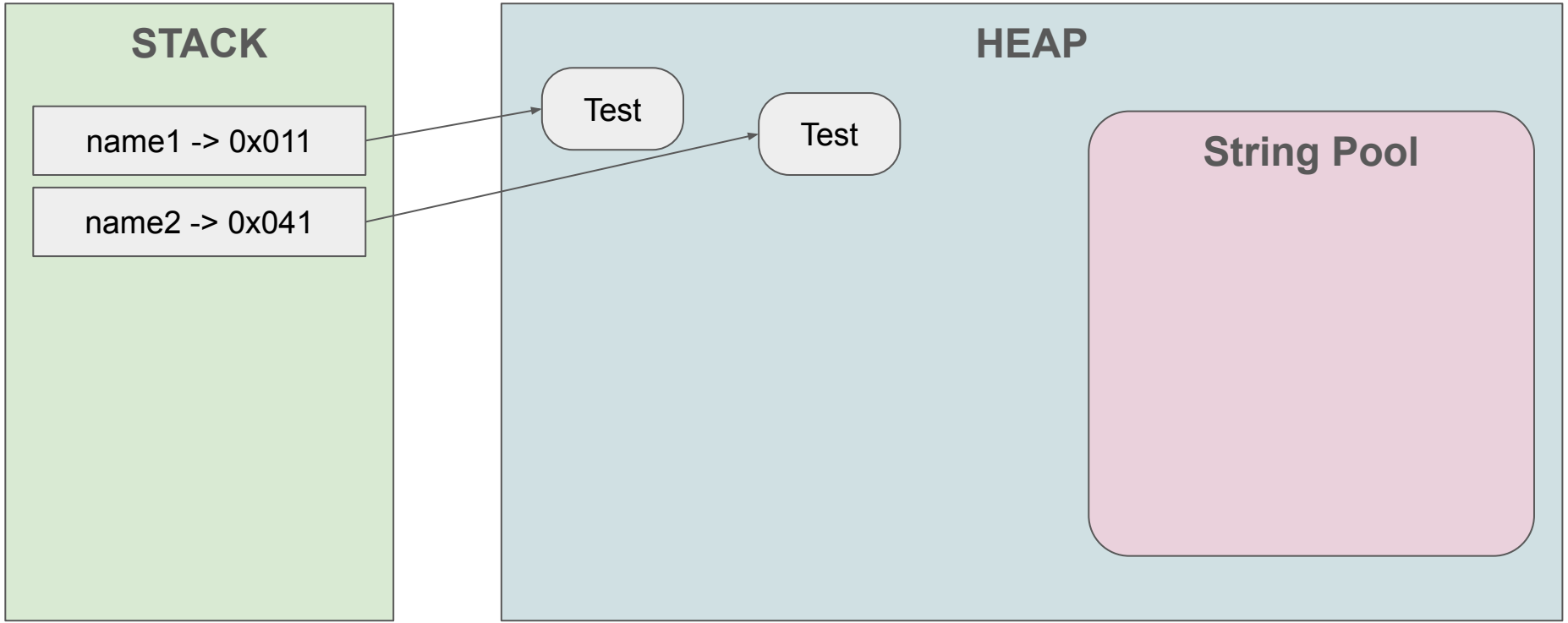
```
String name1 = new String("Test");
```



Java String

agilaghamirzayev

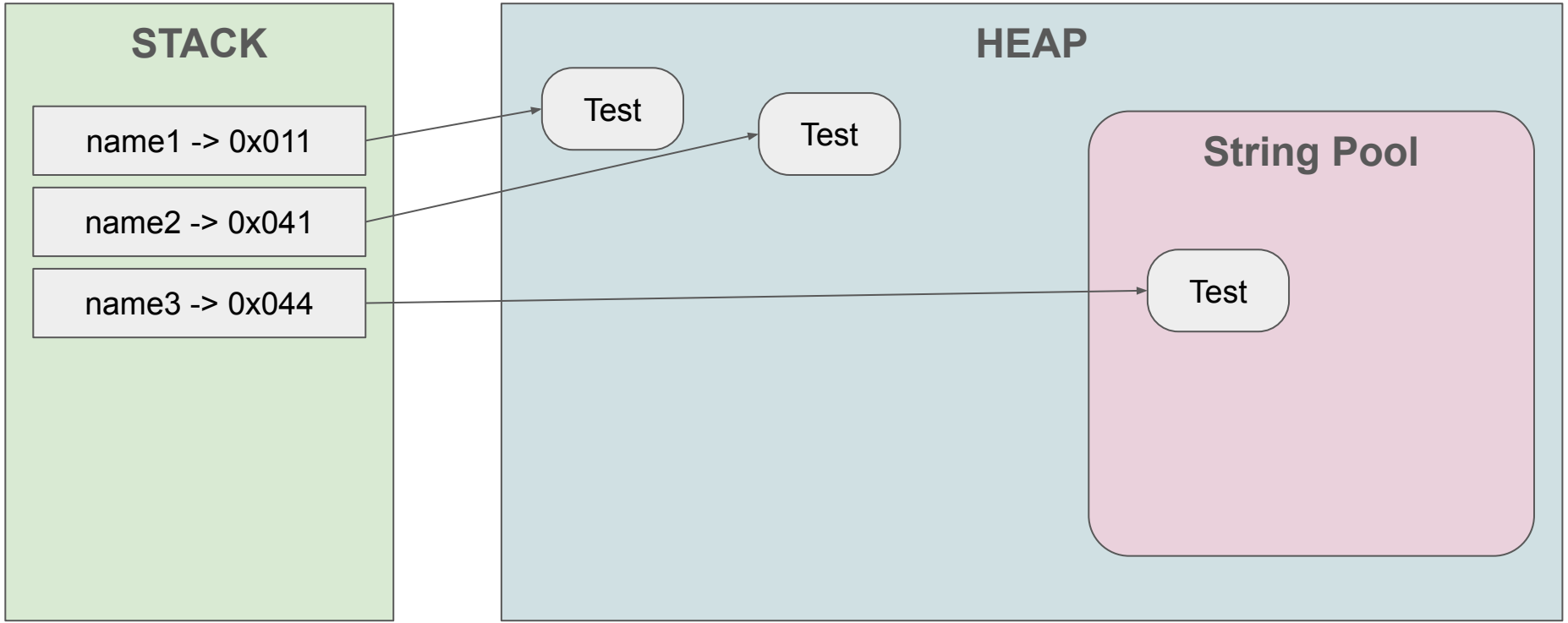
```
String name1 = new String("Test");
String name2 = new String("Test");
```



Java String

agilaghamirzayev

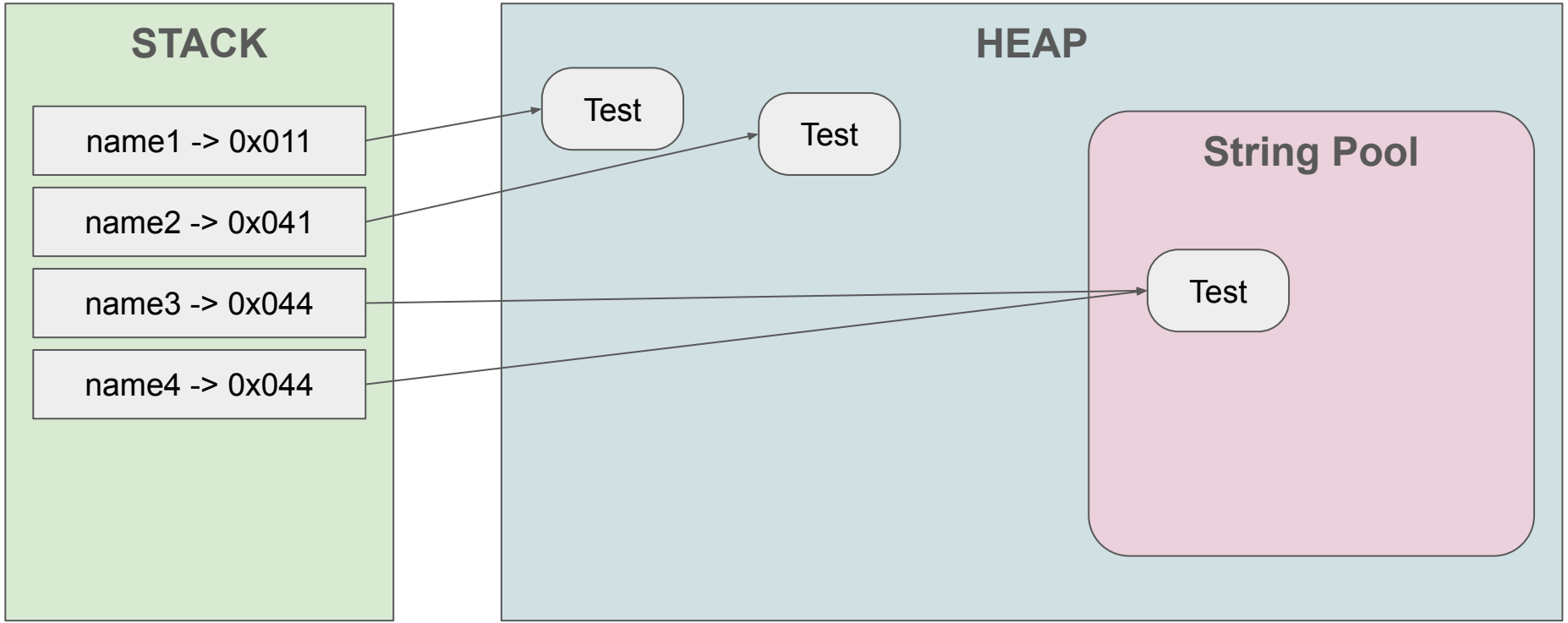
```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
```



Java String

agilaghamirzayev

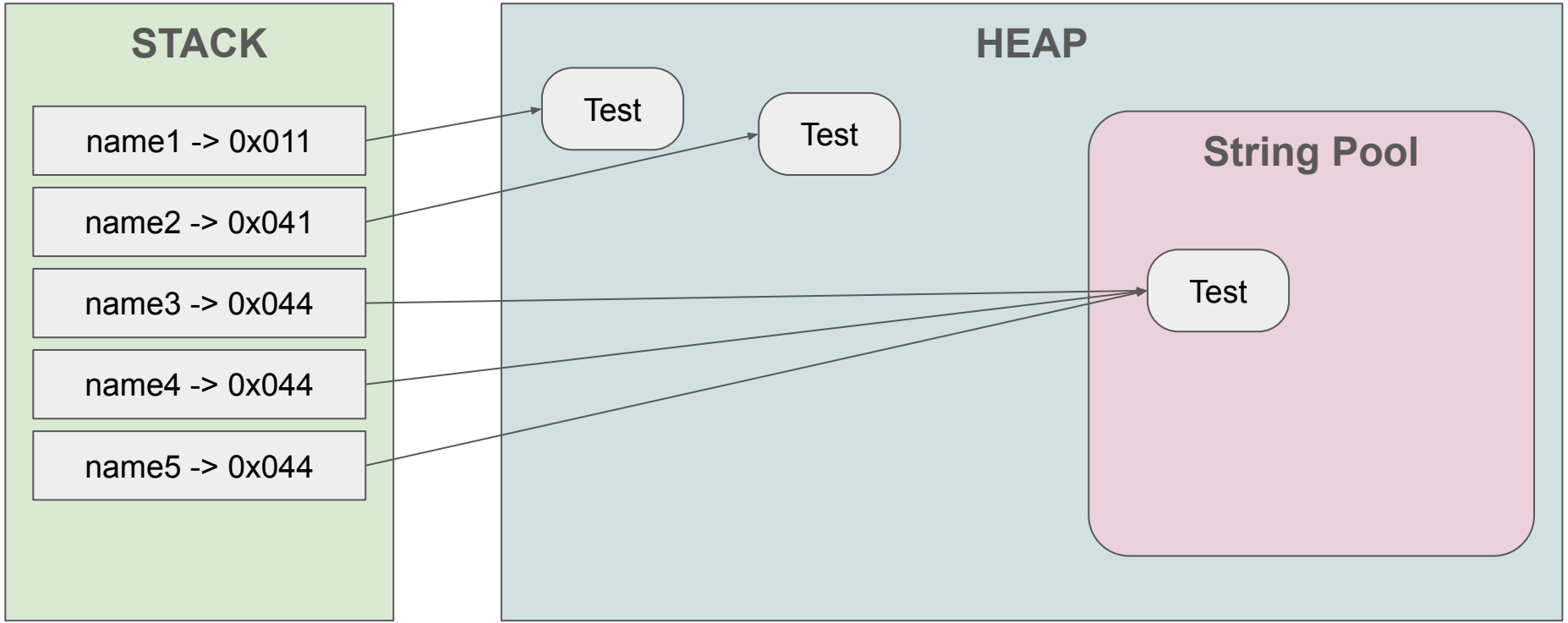
```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
```



Java String

agilaghamirzayev

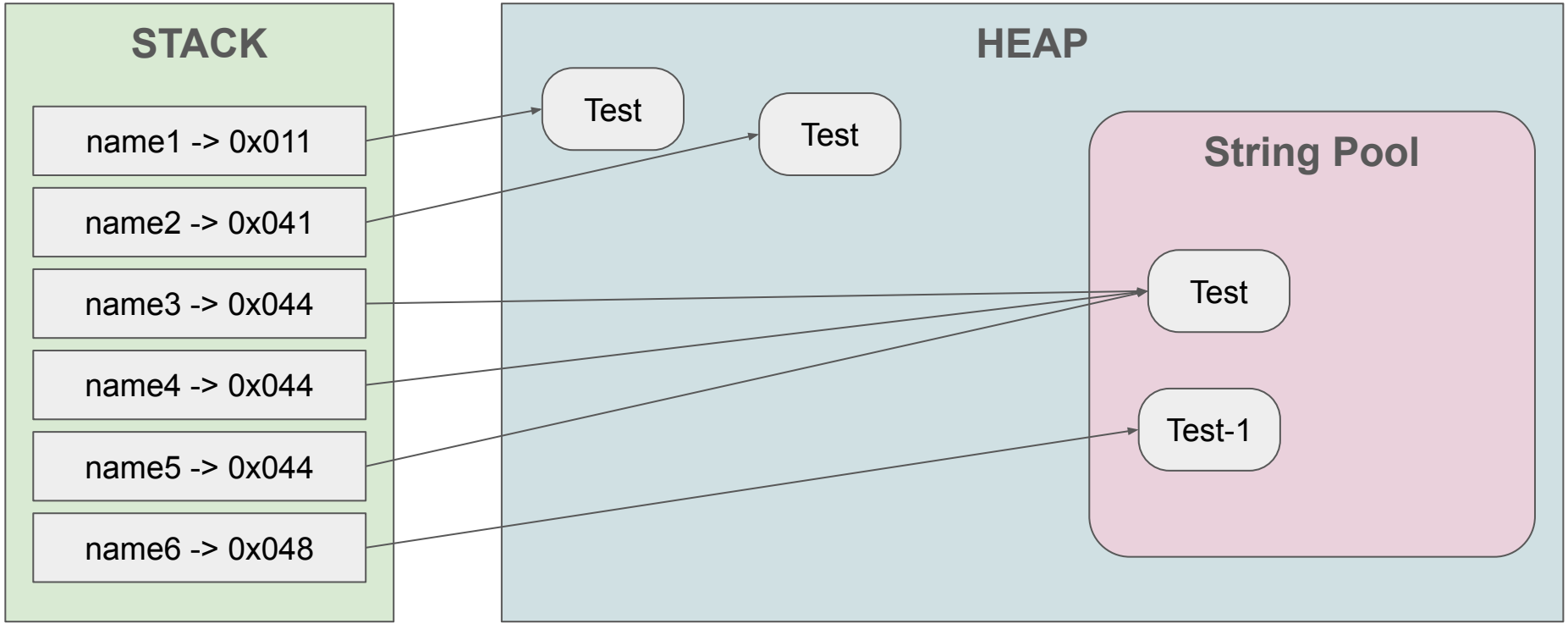
```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
```



Java String

agilaghamirzayev

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

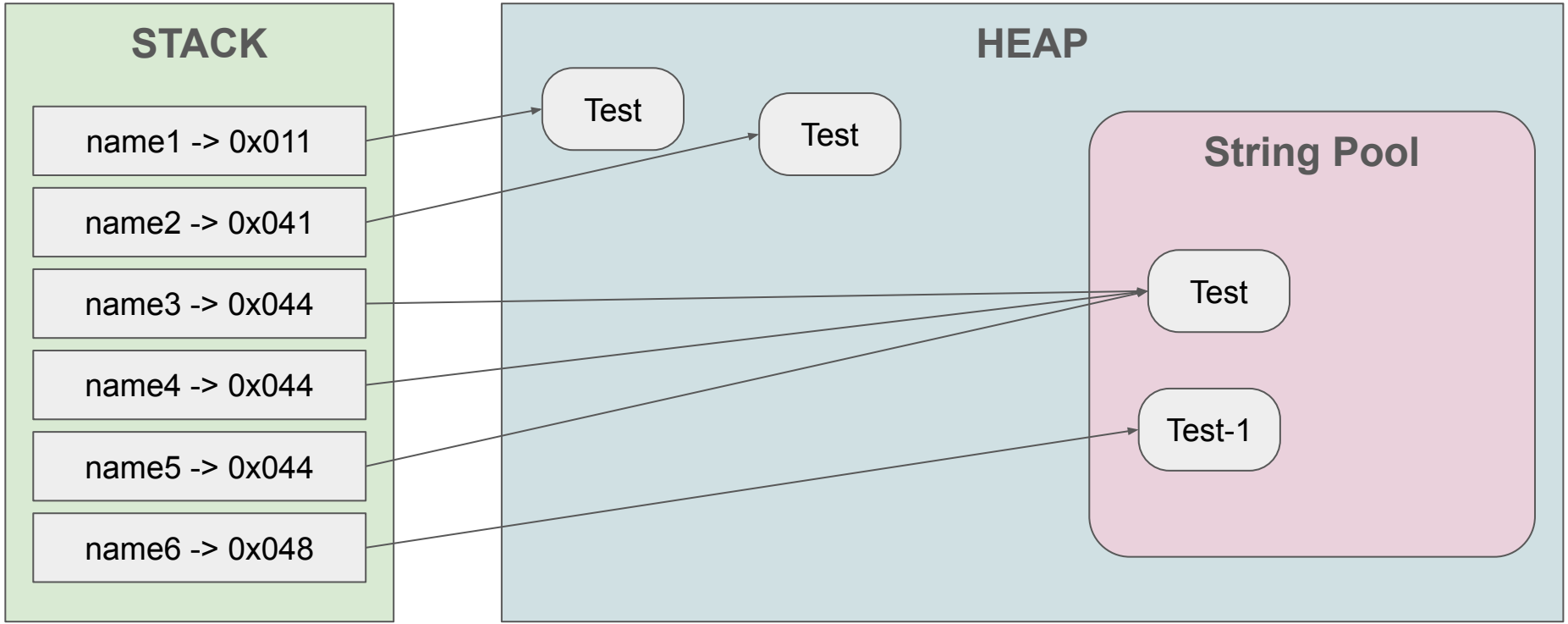


Java String

agilaghamirzayev

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

```
System.out.println(name1 == name2);  // false
System.out.println(name1 == name3);  // false
System.out.println(name3 == name4);  // true
System.out.println(name3 == name5);  // true
System.out.println(name3 == name6);  // false
System.out.println(name1 == name5);  // ????
```

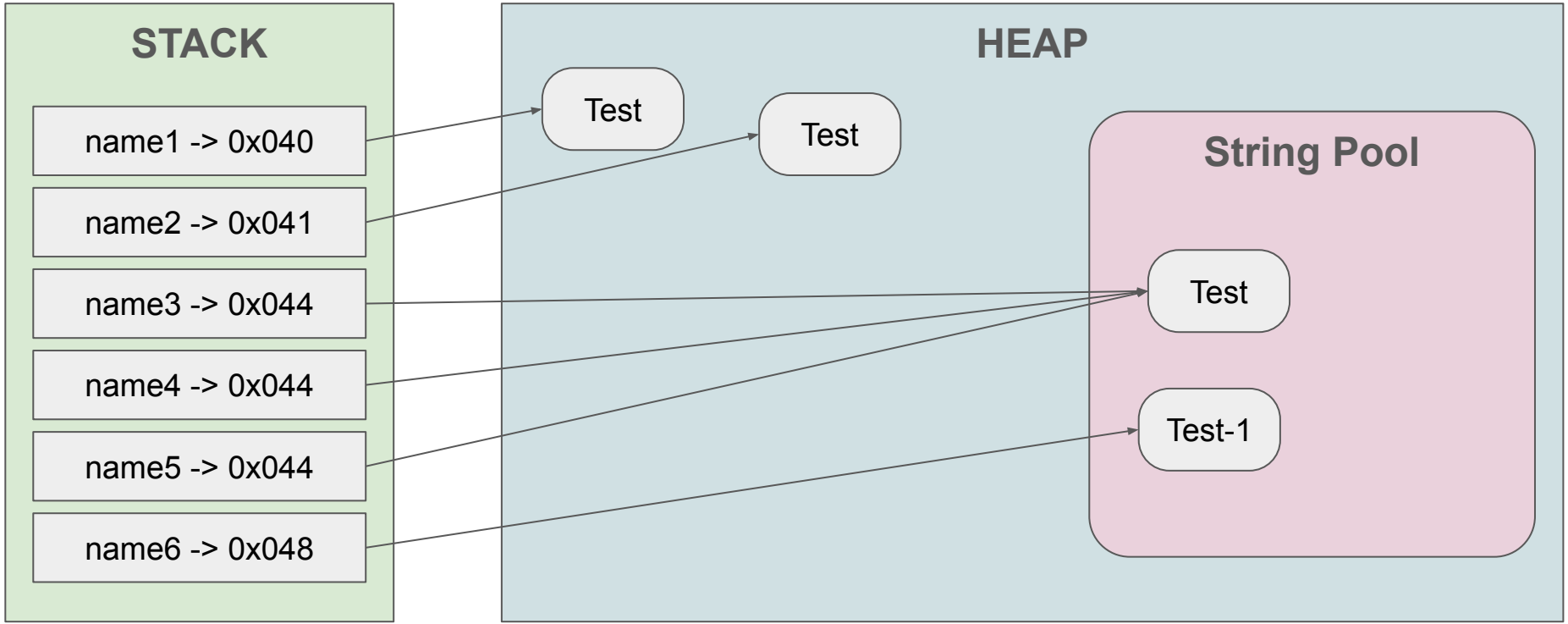


Java String

agilaghamirzayev

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

```
System.out.println(name1 == name2);  // false
System.out.println(name1 == name3);  // false
System.out.println(name3 == name4);  // true
System.out.println(name3 == name5);  // true
System.out.println(name3 == name6);  // false
System.out.println(name1 == name5);  // false
```

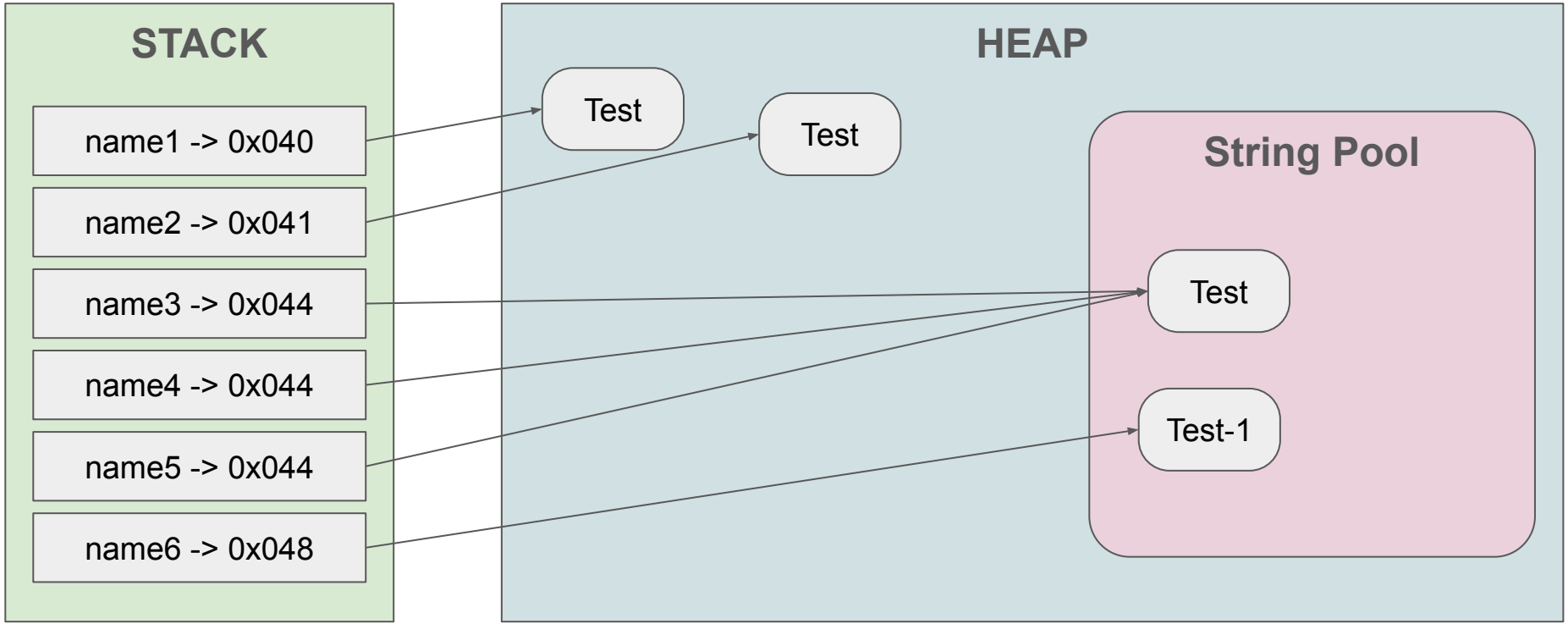


Java String

agilaghamirzayev

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

```
System.out.println(name1 == name2);  // false
System.out.println(name1 == name3);  // false
System.out.println(name3 == name4);  // true
System.out.println(name3 == name5);  // true
System.out.println(name3 == name6);  // false
System.out.println(name1 == name5);  // false
```



String comparison

agilaghamirzayev

```
System.out.println(name1 == name2);  // false
System.out.println(name1 == name3);  // false
System.out.println(name3 == name4);  // true
System.out.println(name3 == name5);  // true
System.out.println(name3 == name6);  // false
System.out.println(name1 == name5);  // false
```

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

```
System.out.println(name1.equals(name2));  // true
System.out.println(name3.equals(name5));  // true
System.out.println(name3.equals(name4));  // true
System.out.println(name3.equals(name5));  // true
System.out.println(name3.equals(name6));  // false
System.out.println(name1.equals(name5));  // true
```

```
String first = "rat" + 1;
String second = "r" + "a" + "t" + "1";
String third = "r" + "a" + "t" + new String("1");

System.out.println(first == second);           // ?
System.out.println(first == second.intern());  // ?
System.out.println(first == third);            // ?
System.out.println(first == third.intern());   // ?
```

String comparison

agilaghamirzayev

```
System.out.println(name1 == name2);  // false
System.out.println(name1 == name3);  // false
System.out.println(name3 == name4);  // true
System.out.println(name3 == name5);  // true
System.out.println(name3 == name6);  // false
System.out.println(name1 == name5);  // false
```

```
String name1 = new String("Test");
String name2 = new String("Test");
String name3 = "Test";
String name4 = "Test";
String name5 = name1.intern();
String name6 = "Test-1";
```

```
System.out.println(name1.equals(name2));  // true
System.out.println(name3.equals(name5));  // true
System.out.println(name3.equals(name4));  // true
System.out.println(name3.equals(name5));  // true
System.out.println(name3.equals(name6));  // false
System.out.println(name1.equals(name5));  // true
```

```
String first = "rat" + 1;
String second = "r" + "a" + "t" + "1";
String third = "r" + "a" + "t" + new String("1");

System.out.println(first == second);           // true
System.out.println(first == second.intern());  // true
System.out.println(first == third);            // false
System.out.println(first == third.intern());   // true
```

String Concatenating


```
System.out.println(1 + 2);          // 3
System.out.println("a" + "b");      // ab
System.out.println("a" + "b" + 3); // ab3
System.out.println(1 + 2 + "c");    // 3c
System.out.println("c" + 1 + 2);    // c12
System.out.println("c" + null);     // cnull
```

- If both operands are numeric, + means numeric addition.
- If either operand is a String, + means concatenation.
- The expression is evaluated left to right.

String Concatenating

agilaghamirzayev

```
System.out.println(1 + 2);          // 3
System.out.println("a" + "b");      // ab
System.out.println("a" + "b" + 3); // ab3
System.out.println(1 + 2 + "c");    // 3c
System.out.println("c" + 1 + 2);    // c12
System.out.println("c" + null);     // cnull
```

- If both operands are numeric, + means numeric addition.
- If either operand is a String, + means concatenation.
- The expression is evaluated left to right.

```
int three = 3;
String four = "4";
System.out.println(1 + 2 + three + four);
```

String Methods


```
String name = "animals";

System.out.println(name.length()); // 7
```

The method length() **returns the number of characters** in the String.
The method signature is as follows:
`public int length()`

String Methods


```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception
```

The method charAt() lets you query the string to **find out what character is at a specific index**.
The method signature is as follows:

```
public char charAt(int index)
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

- The method indexOf() looks at the characters in the string and f**inds the first index that matches the desired value**.
- The indexOf method can work with an individual character or a whole String as input.
- It can also start from a requested position.
- Remember that a char can be passed to an int parameter type.

The method signatures are as follows:

```
public int indexOf(int ch)
public int indexOf(int ch, int fromIndex)
public int indexOf(String str)
public int indexOf(String str, int fromIndex)
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception
```

The method substring() also looks for characters in a string. **It returns parts of the string.**
The first parameter is the index to start with for the returned string. As usual, this is a zero-based index.
There is an optional second parameter, which is the end index you want to stop at.

```
public String substring(int beginIndex)
public String substring(int beginIndex, int endIndex)
```

String Methods

agilaghamirzayev

```java
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```java
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

```
System.out.println("abc".equals("ABC"));           // false
System.out.println("ABC".equals("ABC"));           // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

```
System.out.println("abc".equals("ABC"));           // false
System.out.println("ABC".equals("ABC"));           // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true

System.out.println("abc".startsWith("a")); // true
System.out.println("abc".startsWith("A")); // false
System.out.println("abc".endsWith("c"));   // true
System.out.println("abc".endsWith("a"));   // false
System.out.println("abc".contains("b"));   // true
System.out.println("abc".contains("B"));   // false
```

Again, nothing surprising here. Java is doing a case-sensitive

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

```
System.out.println("abc".equals("ABC"));          // false
System.out.println("ABC".equals("ABC"));          // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true

System.out.println("abc".startsWith("a")); // true
System.out.println("abc".startsWith("A")); // false
System.out.println("abc".endsWith("c"));   // true
System.out.println("abc".endsWith("a"));   // false
System.out.println("abc".contains("b"));   // true
System.out.println("abc".contains("B"));   // false
```

Again, nothing surprising here. Java is doing a case-sensitive

```
System.out.println("abcabc".replace('a', 'A')); // AbcAbc
System.out.println("abcabc".replace("a", "A")); // AbcAbc
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

```
System.out.println("abc".equals("ABC"));          // false
System.out.println("ABC".equals("ABC"));          // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true

System.out.println("abc".startsWith("a")); // true
System.out.println("abc".startsWith("A")); // false
System.out.println("abc".endsWith("c"));   // true
System.out.println("abc".endsWith("a"));   // false
System.out.println("abc".contains("b"));   // true
System.out.println("abc".contains("B"));   // false
```

Again, nothing surprising here. Java is doing a case-sensitive

```
System.out.println("abcabc".replace('a', 'A')); // AbcAbc
System.out.println("abcabc".replace("a", "A")); // AbcAbc

System.out.println(" ".isEmpty()); // false
System.out.println("".isEmpty());  // true
System.out.println(" ".isBlank()); // true
System.out.println("".isBlank());  // true
```

String Methods

agilaghamirzayev

```
String name = "animals";

System.out.println(name.length()); // 7

System.out.println(name.charAt(0)); // a
System.out.println(name.charAt(6)); // s
System.out.println(name.charAt(7)); // exception

System.out.println(name.indexOf('a'));      // 0
System.out.println(name.indexOf("al"));     // 4
System.out.println(name.indexOf('a', 4));   // 4
System.out.println(name.indexOf("al", 5));  // - 1
```

```
System.out.println(name.substring(3));                // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
System.out.println(name.substring(3, 4));             // m
System.out.println(name.substring(3, 7));             // mals

System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception

System.out.println(name.toUpperCase());     // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

```
System.out.println("abc".equals("ABC"));           // false
System.out.println("ABC".equals("ABC"));           // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true

System.out.println("abc".startsWith("a")); // true
System.out.println("abc".startsWith("A")); // false
System.out.println("abc".endsWith("c"));   // true
System.out.println("abc".endsWith("a"));   // false
System.out.println("abc".contains("b"));   // true
System.out.println("abc".contains("B"));   // false
```

Again, nothing surprising here. Java is doing a case-sensitive

```
System.out.println("abcabc".replace('a', 'A')); // AbcAbc
System.out.println("abcabc".replace("a", "A")); // AbcAbc

System.out.println(" ".isEmpty()); // false
System.out.println("".isEmpty());  // true
System.out.println(" ".isBlank()); // true
System.out.println("".isBlank());  // true

String text = " abc\t ";
System.out.println(text.trim().length());          // 3
System.out.println(text.strip().length());         // 3
System.out.println(text.stripLeading().length()); // 5
System.out.println(text.stripTrailing().length()); // 4
```

String Methods

agilaghamirzayev

```
String name = "Kate";
int orderId = 5;

// All print: Hello Kate, order 5 is ready
System.out.println("Hello "+ name + ", order " + orderId + " is ready");
System.out.println(String.format("Hello %s, order %d is ready", name, orderId));
System.out.println("Hello %s, order %d is ready".formatted(name, orderId));
```

%s *Applies to any type, commonly String values*
%d *Applies to integer values like int and long*
%f *Applies to floating- point values like float and double*
%n *Inserts a line break using the system- dependent line separator*

String Methods

agilaghamirzayev

```
String name = "Kate";
String orderId = 5;

// All print: Hello Kate, order 5 is ready
System.out.println("Hello "+ name + ", order " + orderId + " is ready");
System.out.println(String.format("Hello %s, order %d is ready", name, orderId));
System.out.println("Hello %s, order %d is ready".formatted(name, orderId));
```

```
String fruits = "apple,orange,banana,grape";
String[] fruitArray = fruits.split(",");
```

String Methods

agilaghamirzayev

```
String name = "Kate";
String orderId = 5;

// All print: Hello Kate, order 5 is ready
System.out.println("Hello "+ name + ", order " + orderId + " is ready");
System.out.println(String.format("Hello %s, order %d is ready", name, orderId));
System.out.println("Hello %s, order %d is ready".formatted(name, orderId));
```

```
String fruits = "apple,orange,banana,grape";
String fruitArray[] = fruits.split(",");
```

```
String result = String.join(delimiter, elements);
String result = String.join(" ", "Hello", "World", "Java"); // Hello World Java
```

StringBuilder

agilaghamirzayev

```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

StringBuilder


```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

How many objects do you think this piece of code creates?

StringBuilder


```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

How many objects do you think this piece of code creates?

27 objects

StringBuilder

agilaghamirzayev

```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

```
StringBuilder alpha = new StringBuilder();
for(char current = 'a'; current <= 'z'; current++)
    alpha.append(current);
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

StringBuilder vs String

agilaghamirzayev

```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

```
StringBuilder alpha = new StringBuilder();
for(char current = 'a'; current <= 'z'; current++)
    alpha.append(current);
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

```
StringBuilder a = new StringBuilder("abc");
StringBuilder b = a.append("de");
b = b.append("f").append("g");
System.out.println("a=" + a); //abcdefg
System.out.println("b=" + b); //abcdefg
```

```
String a = new String("abc");
String b = a + "de";
b = b + "f" + "g";
System.out.println("a=" + a); //abc
System.out.println("b=" + b); //abcdefg
```

StringBuilder vs String

agilaghamirzayev

```
String alpha = "";
for(char current = 'a'; current <= 'z'; current++)
    alpha += current;
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

```
StringBuilder alpha = new StringBuilder();
for(char current = 'a'; current <= 'z'; current++)
    alpha.append(current);
System.out.println(alpha); // abcdefghijklmnopqrstuvwxyz
```

```
StringBuilder a = new StringBuilder("abc");
StringBuilder b = a.append("de");
b = b.append("f").append("g");
System.out.println("a=" + a); //abcdefg
System.out.println("b=" + b); //abcdefg
```

```
String a = new String("abc");
String b = a + "de";
b = b + "f" + "g";
System.out.println("a=" + a); //abc
System.out.println("b=" + b); //abcdefg
```

Why Strings Are Designed to Be Immutable

- **Memory Efficiency:** The String Pool allows multiple references to share the same string object safely.
- **Thread Safety:** Immutable objects are inherently safe for multi-threaded access.
- **Hashcode Reliability:** Strings are commonly used as keys in HashMap; immutability ensures the hashcode remains consistent.
- **Performance Optimization:** JVM can optimize immutable strings, including interning, which saves memory and improves speed.

StringBuilder vs StringBuffer

agilaghamirzayev

Feature	StringBuilder	StringBuffer
Thread safety	Not thread-safe	Thread-safe
Performance	Faster	Slower
Synchronization	No	Yes (synchronized)
Use case	Single-thread	Multi-thread

```
// Create an empty StringBuilder
StringBuilder sb = new StringBuilder();
sb.append("World");       // World
sb.insert(o, "Hello");    // Hello World
sb.deleteCharAt(4);       // Hell World
sb.insert (4, 'o');       // Hello World
sb.setLength(5);          // Hello
sb.reverse();             // olleH
```

```
// Create an empty StringBuffer
StringBuffer sb = new StringBuffer();
sb. append ("World");    // World
sb.insert(o, "Hello "); // Hello World
sb.deleteCharAt(4);      // Hell World
sb.insert(4, 'o');      // Hello World
sb.setLength(5);         // Hello
sb.reverse();            // olleH
```

Used Sources

Books:

Oracle® Certified Professional Java SE 17 Developer (Scott Selikoff, Jeanne Boyarsky)

Sites:

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html
https://www.w3schools.com/java/java_strings.asp
https://medium.com/data-structures-and-algorithms-series/java-arrays-4cb311c3ad5f
https://www.geeksforgeeks.org/java/java-string-is-immutable-what-exactly-is-the-meaning/
https://chatgpt.com/